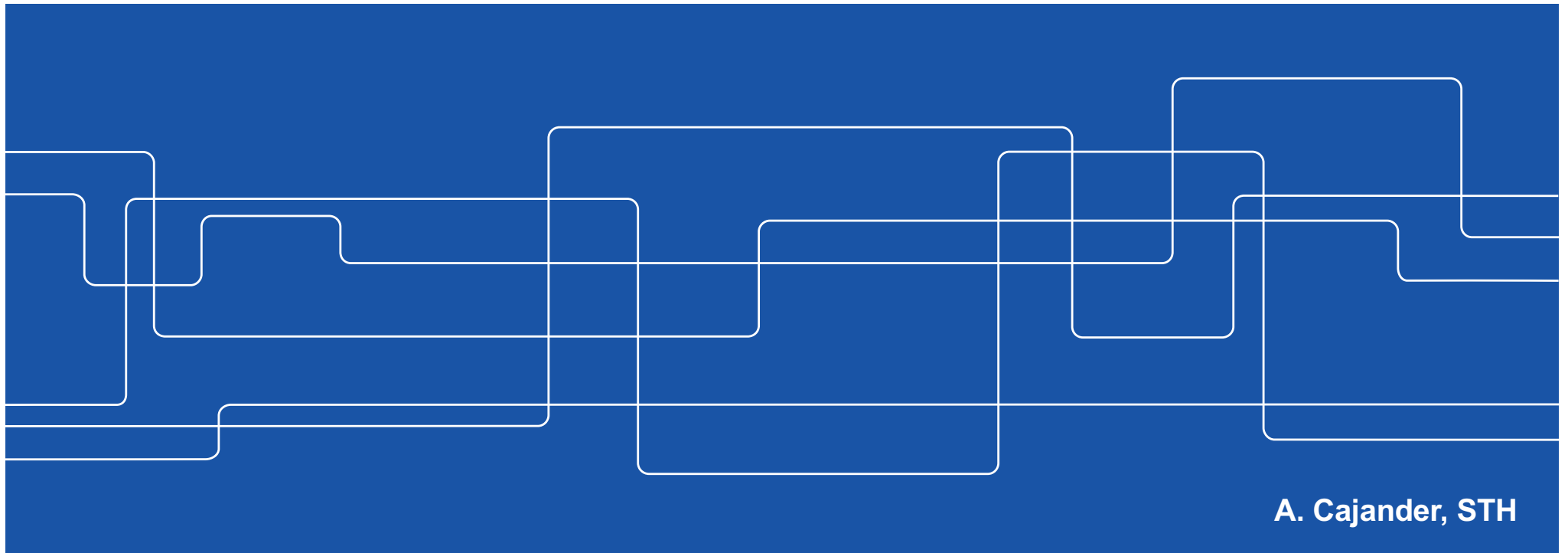




HE1025 Operativsystem

L2: Paging



A. Cajander, STH



Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

F2: V#3 Schemaläggning

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: V#6 Buffer

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 File/DMA

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

Kap 7..9,(10-11), RTOS

Lab #1 Handledning

Kap 12..13, 15..17

t.b.d

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

.ppt

Kap (47, 39) & 48, (49..51)

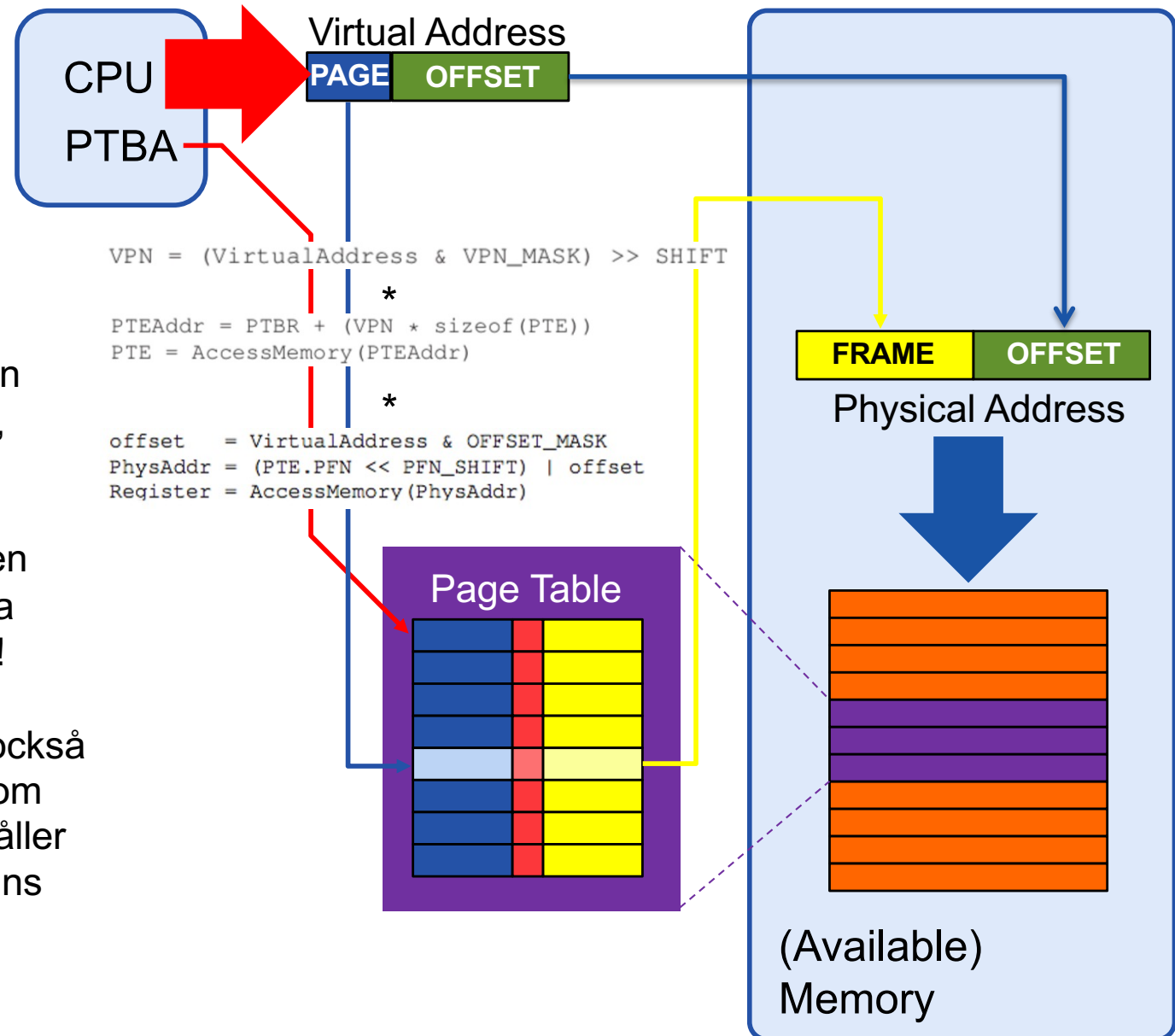
Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



Paging



Den virtuella adressrymden delas upp i lika stora sidor, typiskt 4kByte stora...

...det fysiska adressrymden delas också upp i lika stora sidor men de kallas ramar!

I det fysiska minnet finns också en sidtabell per process som talar om vilken ram som håller en virtuell sida (om den finns etc...).

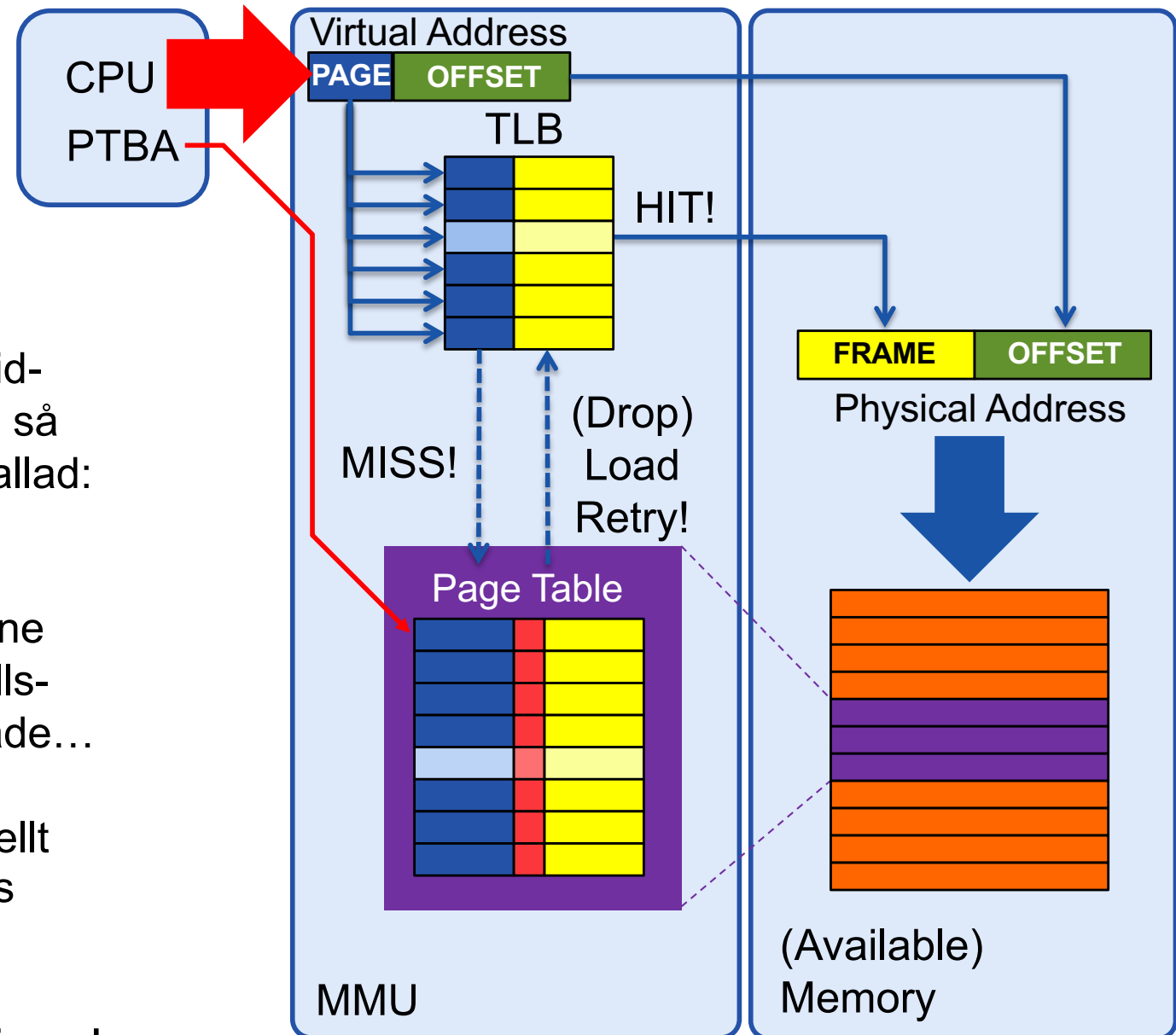
Vid varje minnesaccess översätts sidadressen till en ramadress med hjälp av sidtabellen! (Kruks: Tar tid & linjär sidtabell vanligtvis jättestor!)

Paging

För att hantera att en sidtabellsindexering tar tid så finns i dag ofta en så kallad: Translation Look-aside Buffer (TLB) som är ett separat associativt minne där de senaste sidtabellsökningarna finns lagrade...

...minnet avsöks parallellt och finns en träff erhålls svaret direkt...

...vid miss, gör som tidigare!



Utan TLB-tricket hade vi inte använt sidtekniken

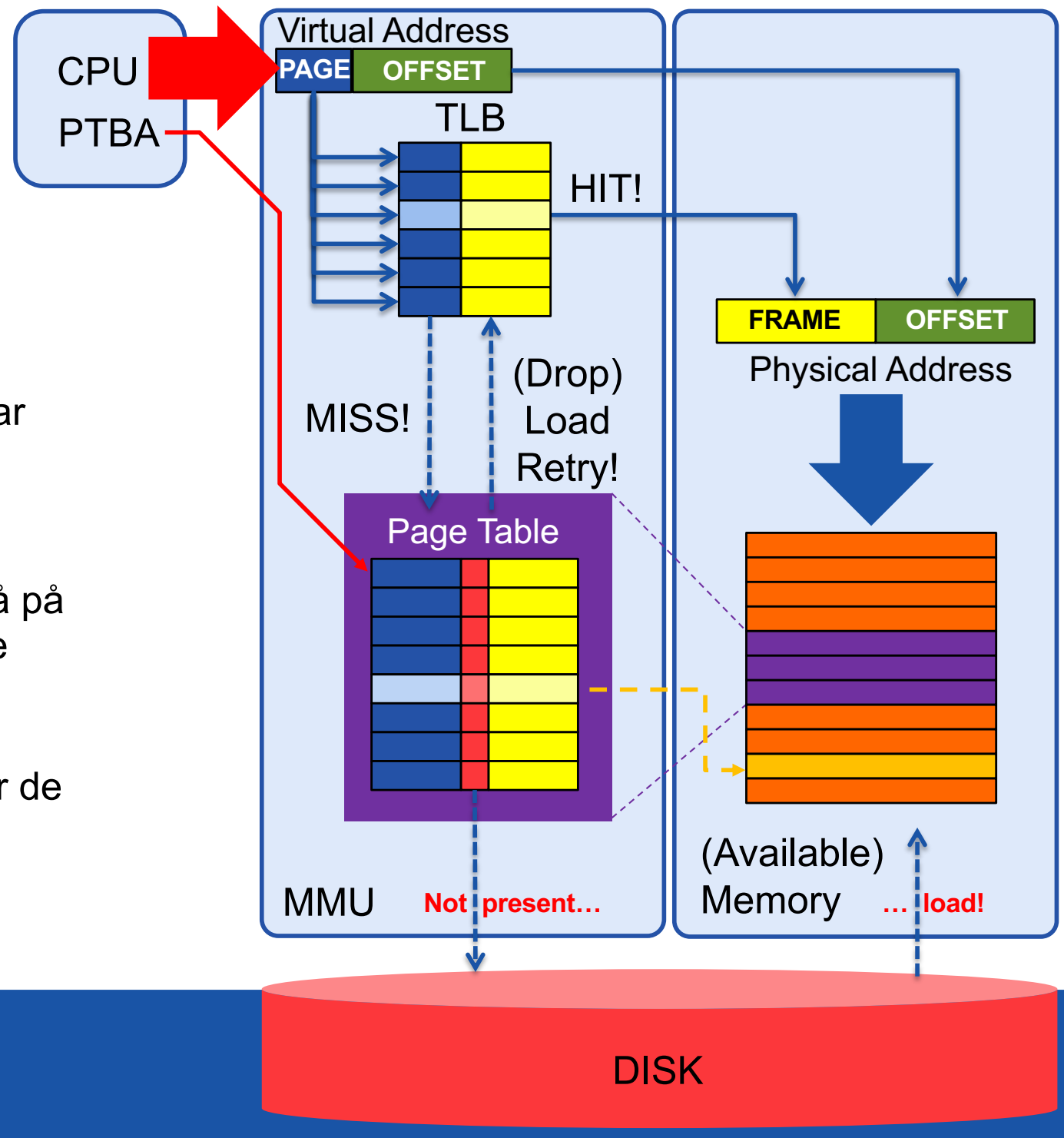


Paging

Det fysiska minnet klarar
sällan att hålla alla
processers sidor...

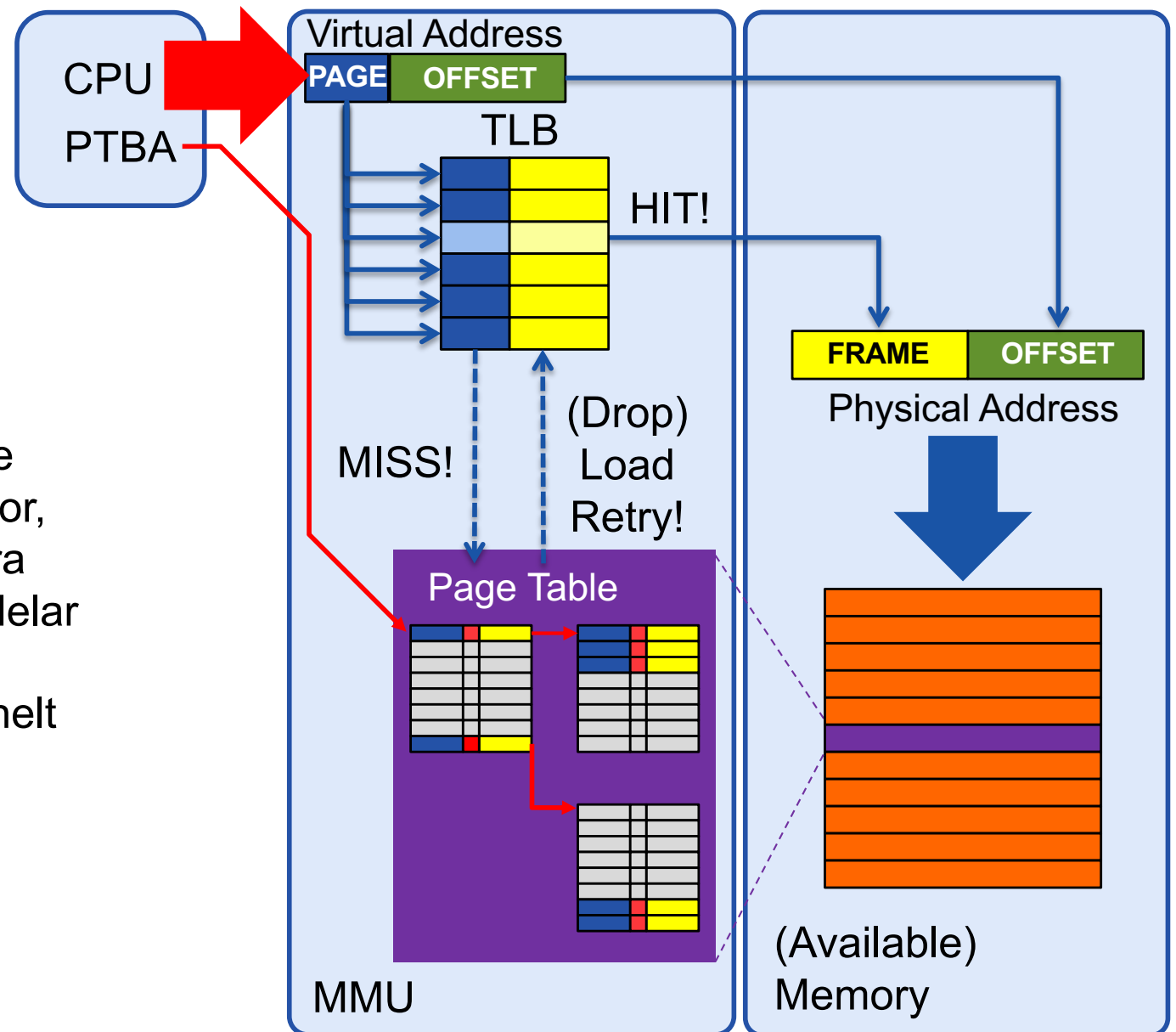
...en del sidor lagras då på
datorns hårddisk tills de
behövs...

...när det börjar ske går de
flesta datorer märkbart
långsammare!



Paging

Problemet med att varje (linjär) Page Table är stor, hanteras genom att göra den hierarkisk, där de delar som innehåller enbart förbjudna mappningar helt enkelt kan uteslutas!





NS32082-10 Memory Management Unit

General Description

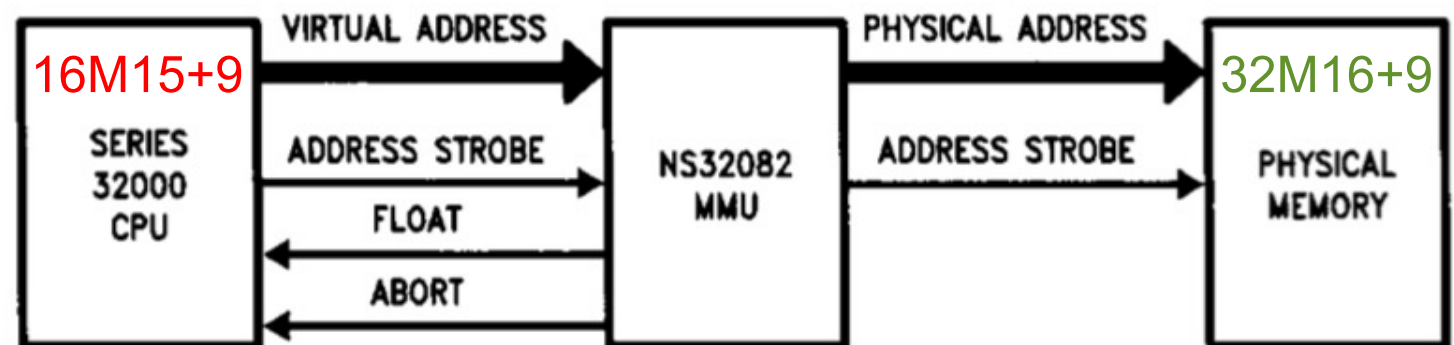
The NS32082 Memory Management Unit (MMU) provides hardware support for demand-paged virtual memory implementations. The NS32082 functions as a slave processor in Series 32000 microprocessor-based systems. Its specific capabilities include fast dynamic translation, protection, and detailed status to assist an operating system in efficiently managing up to 32 Mbytes of physical memory. Support for multiple address spaces, virtual machines, and program debugging is provided.

High-speed address translation is performed on-chip through a 32-entry fully associative translation look-aside buffer (TLB), which maintains itself from tables in memory with no software intervention. Protection violations and page faults (references to non-resident pages) are automatically detected by the MMU, which invokes the instruction abort feature of the CPU.

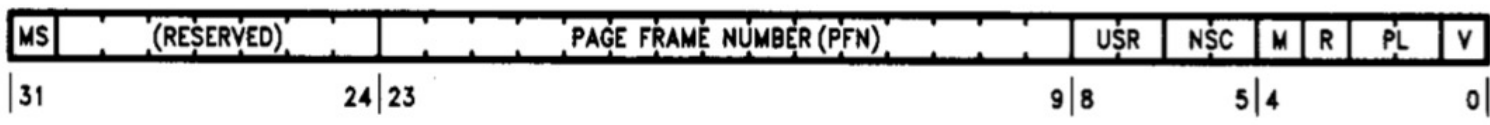
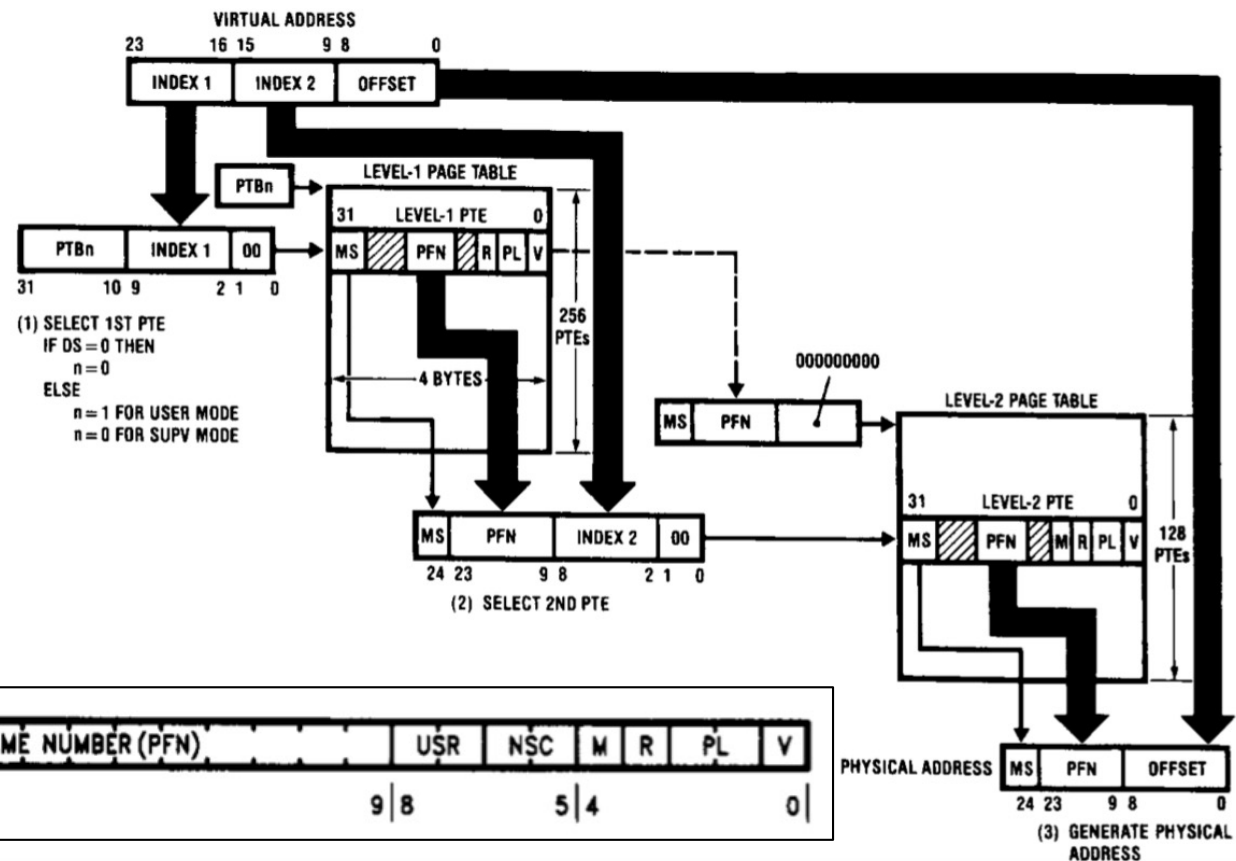
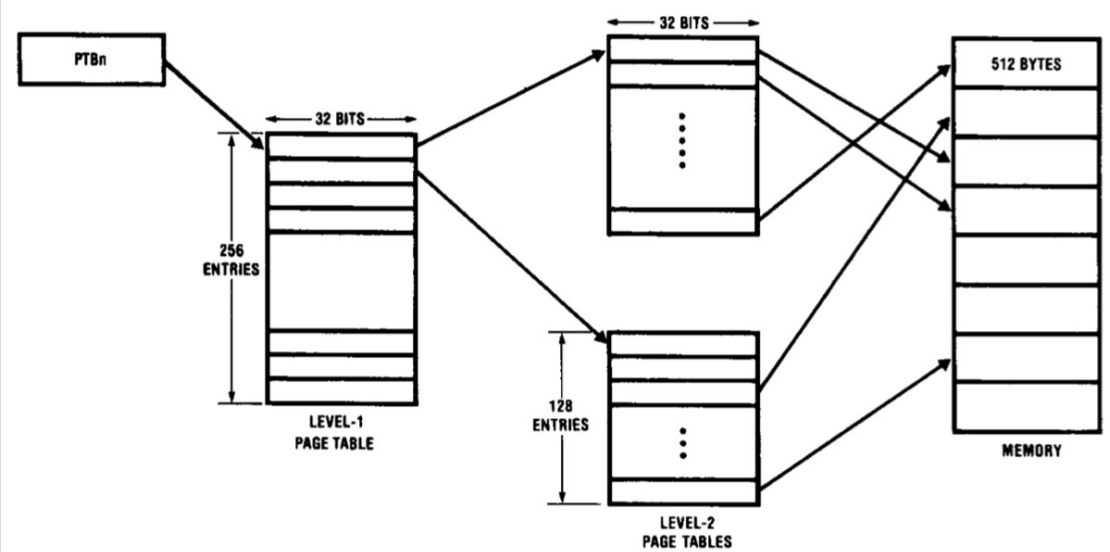
Additional features for program debugging include two breakpoint registers and a breakpoint counter, which provide the programmer with powerful stand-alone debugging capability.

Features

- Totally automatic mapping of 16 Mbyte virtual address space using memory based tables
- On-chip translation look-aside buffer allows 97% of translations to occur in one clock for most applications
- Full hardware support for virtual memory and virtual machines
- Implements "referenced" bits for simple, efficient working set management
- Protection mechanisms implemented via access level checking and dual space mapping
- Program debugging support
- Compatible with NS32016, NS32032 and NS32332 CPUs
- 48-pin dual-in-line package



TAG		VALUE		
AS	PAGE NUMBER (15 BITS)	PL	M	TRANSLATION (16 BITS)
0	xxx	11	0	mmm
1	yyy	11	0	nnn
0	zzz	11	1	ppp
1	www	00	1	qqq





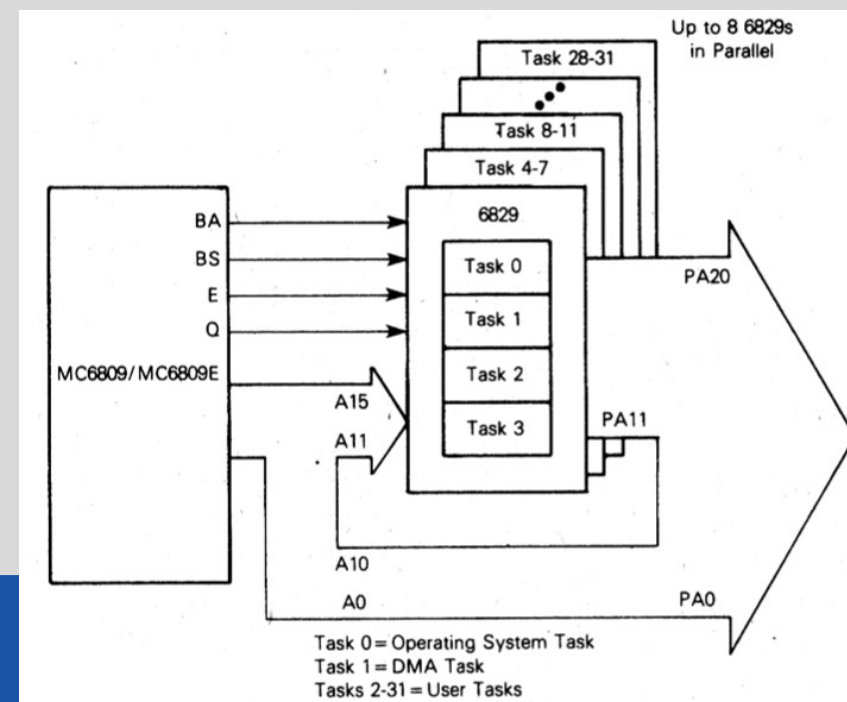
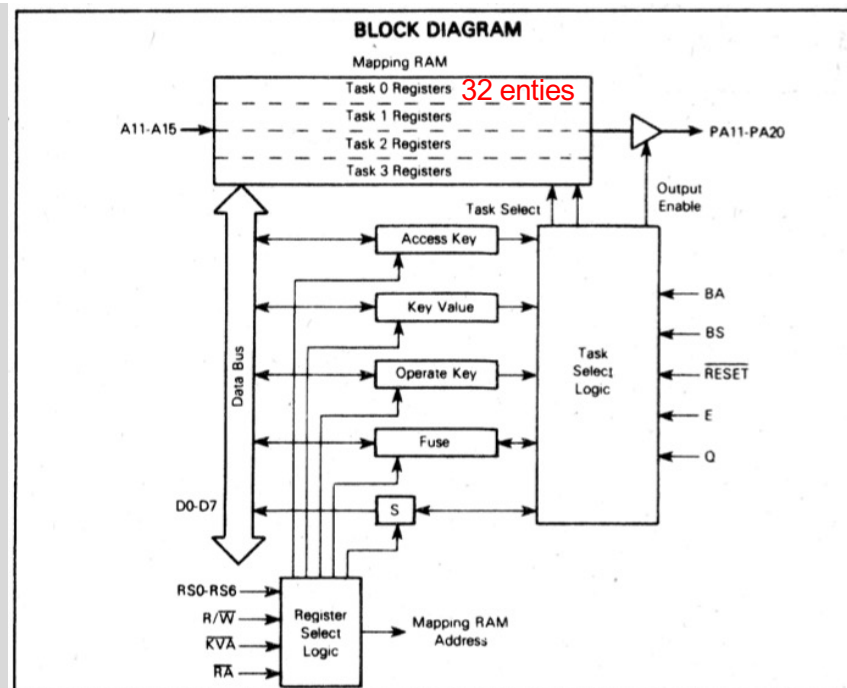
MOTOROLA

Advance Information

MEMORY MANAGEMENT UNIT

The principle function of the MC6829 Memory Management Unit (MMU) is to expand the address space of the MC6809 from 64K bytes to a maximum of 2 Megabytes. Each MMU is capable of handling four different concurrent tasks including DMA. The MMU can also protect the address space of one task from modification by another task. Memory address space expansion is accomplished by applying the upper five address lines of the processor (A11-A15) along with the contents of a 5-bit task register to an internal high-speed mapping RAM. The MMU output consists of ten physical address lines (PA11-PA20) which, when combined with the eleven lower address lines of the processor (A0-A10), forms a physical address space of 2 Megabytes. Each task is assigned memory in increments of 2K bytes up to a total of 64K bytes. In this manner, the address spaces of different tasks can be kept separate from one another. The resulting simplification of the address space programming model will increase the software reliability of a complex multi-process system.

- Expands Memory Address Space from 64K to 2 Megabytes
- Each MMU is Capable of Handling Four Separate Tasks
- Up to Eight MMUs can be Used in a System
- Provides Task Isolation and Write Protection
- Provides Efficient Memory Allocation; 1024 Pages of 2K Bytes Each
- Designed for Efficient Use with DMA
- Fast, Automatic On-Chip Task Switching
- Allows Inter-Process Communication Through Shared Resources
- Simplifies Programming Model of Address Space
- Increases System Software Reliability
- MC6809/MC6800 Bus Compatible
- Single 5-Volt Power Supply





(Omöjlig att implementera.)

Optimal policy (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2*	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

} Cold-start miss aka Compulsory miss

We can also calculate the hit rate for the cache: with 6 hits and 5 misses, the hit rate is $\frac{Hits}{Hits+Misses}$ which is $\frac{6}{6+5}$ or 54.5%. You can also compute the hit rate *modulo* compulsory misses (i.e., ignore the *first* miss to a given page), resulting in a 85.7% hit rate.

* Vi behöver både 0 & 1 innan vi behöver 2 igen, så släng ut 2!



(Lätt att implementera, användes i de första datorerna)

FIFO policy?! (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3
0	Miss	1	First-in→	2, 3, 0
3	Hit		First-in→	2, 3, 0
1	Miss	2	First-in→	3, 0, 1
2	Miss	3	First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2

} Cold-start miss aka Compulsory miss

Comparing FIFO to optimal, FIFO does notably worse: a 36.4% hit rate (or 57.1% excluding compulsory misses). FIFO simply can't determine the importance of blocks: even though page 0 had been accessed a number of times, FIFO still kicks it out, simply because it was the first one brought into memory.

Det är egentligen ännu sämre, det finns sekvenser av sidreferenser, där det i princip alltid blir fel. Problemet benämns Belady's anomali.

Random policy?! (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0
1	Miss	3	2, 0, 1
2	Hit		2, 0, 1
1	Hit		2, 0, 1

Cold-start miss aka
Compulsory miss

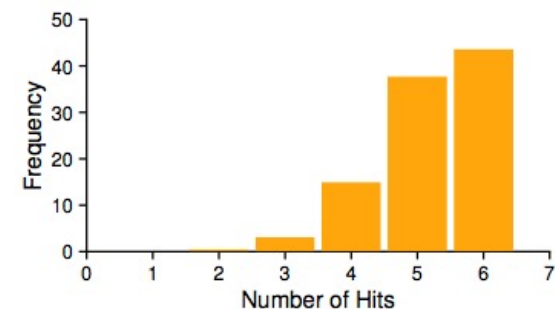


Figure 22.4: Random Performance Over 10,000 Trials

I snitt bättre än FIFO och lite sämre än optimal,
men kan om oturen är framme vara riktigt usel...



(Möjlig att implementera, även om det kräver en hel del administration.)

LRU(LFU) policy?! (for 01201303121 / 3 page cache)

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→ 0	} Cold-start miss aka Compulsory miss
1	Miss		LRU→ 0, 1	
2	Miss		LRU→ 0, 1, 2	
0	Hit		LRU→ 1, 2, 0	
1	Hit		LRU→ 2, 0, 1	
3	Miss	2	LRU→ 0, 1, 3	
0	Hit		LRU→ 1, 3, 0	
3	Hit		LRU→ 1, 0, 3	
1	Hit		LRU→ 0, 3, 1	
2	Miss	0	LRU→ 3, 1, 2	
1	Hit		LRU→ 3, 2, 1	

För denna sekvens alltid lika bra som optimal policy...
...vi kan inte se in i framtiden men tids- och platslokalitet finns ofta i kod!

I verkligheten...

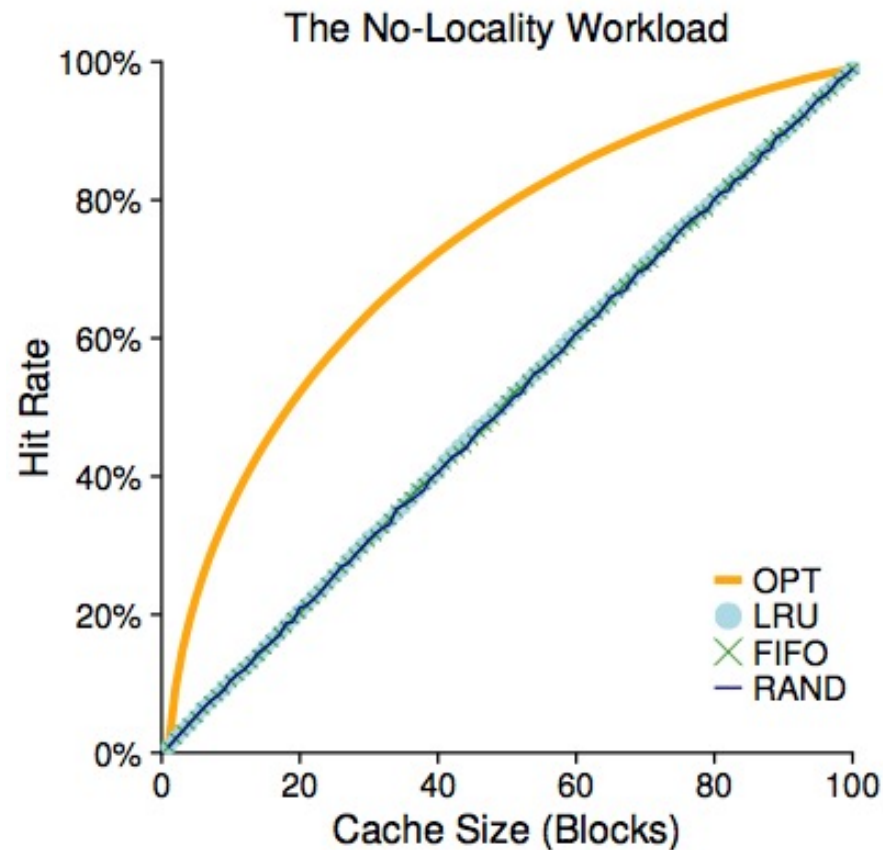


Figure 22.6: The No-Locality Workload

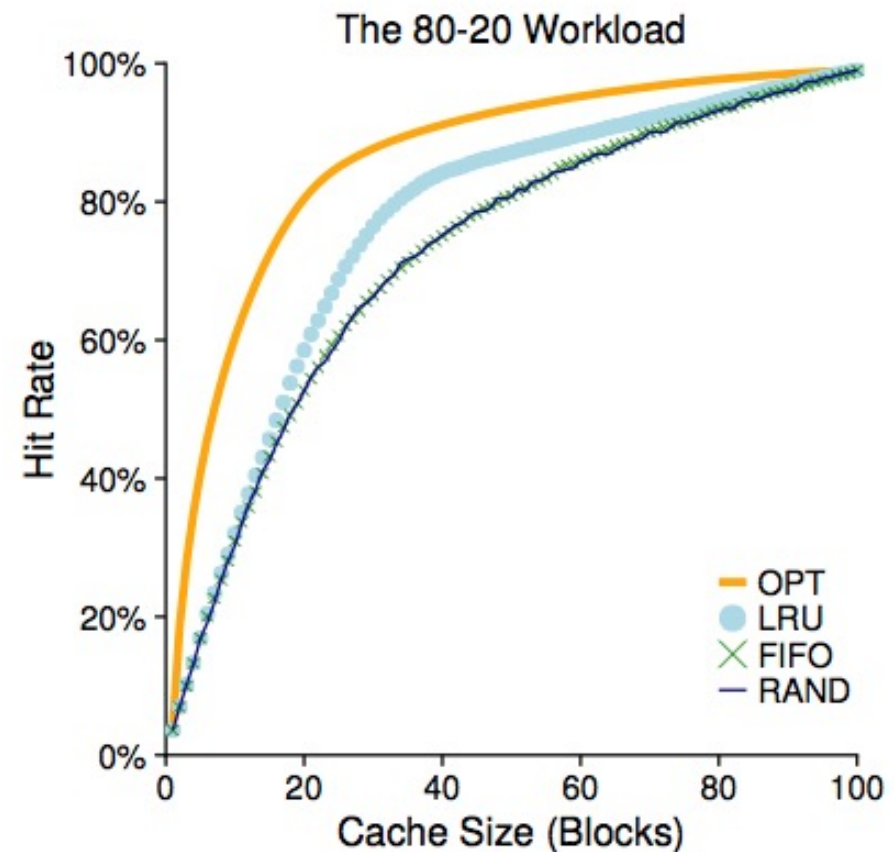


Figure 22.7: The 80-20 Workload

Så fort det finns någon form av lokalitet går det att bli bättre än slumpen!
Kom ihåg att en förbättring på 1/10-dels procent kan göra underverk!!!

Approximating LRU!

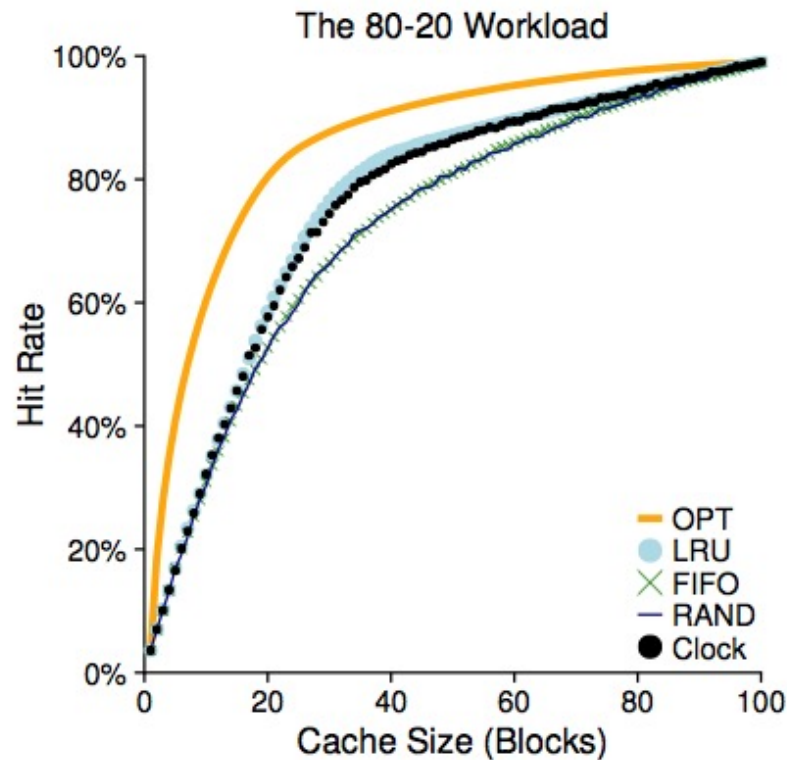


Figure 22.9: The 80-20 Workload With Clock

Vi vill ha en LRU policy men det är svårt i praktiken att tidsstämpla varje access, för att inte tala om problemet att undersöka alla tidsstämplar när en sida ska slängas ut!

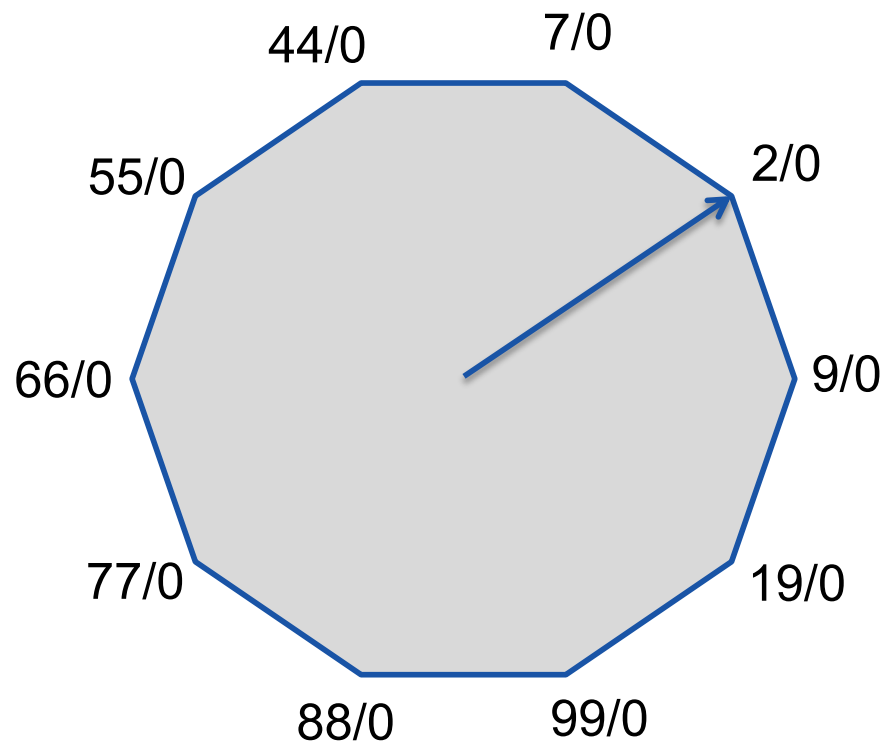
Vi behöver en policy som liknar LRU men är enklare att implementera!

Det finns en sådan app. LRU policy som ofta kallas klockalgoritmen...



Klockalgoritmen

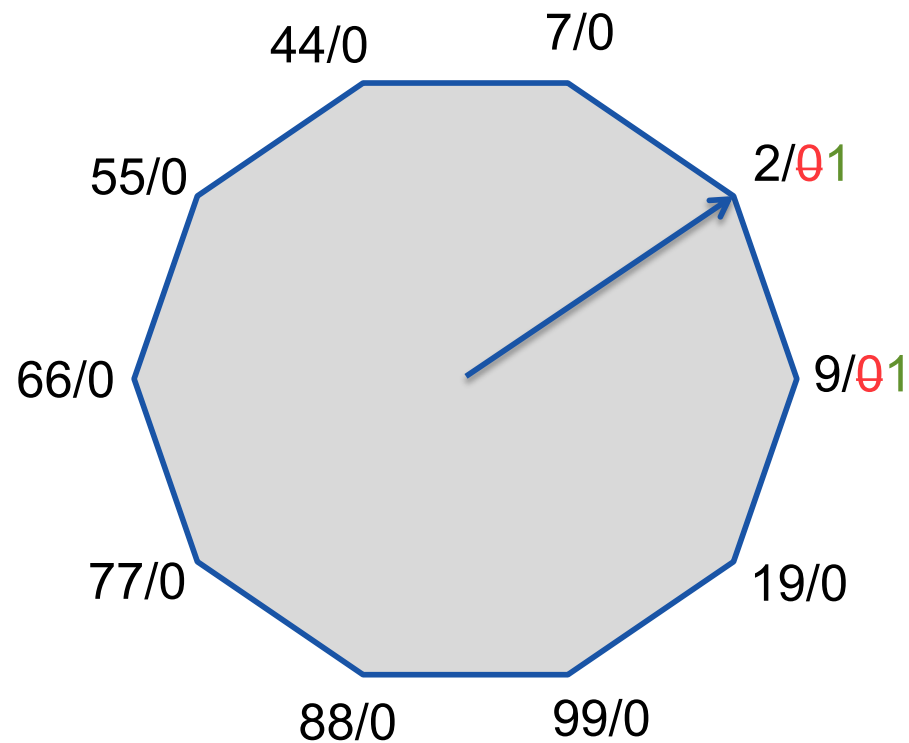
- Tänk dig sidreferenserna ordnade som en cirkulär kö som talen på en urtavla. Varje sida har en referens-bit som initialt sätt till 0...



(Sidorna 7,2,9,19,99,88,77,66,55,44 har nyligen efterfrågats.)



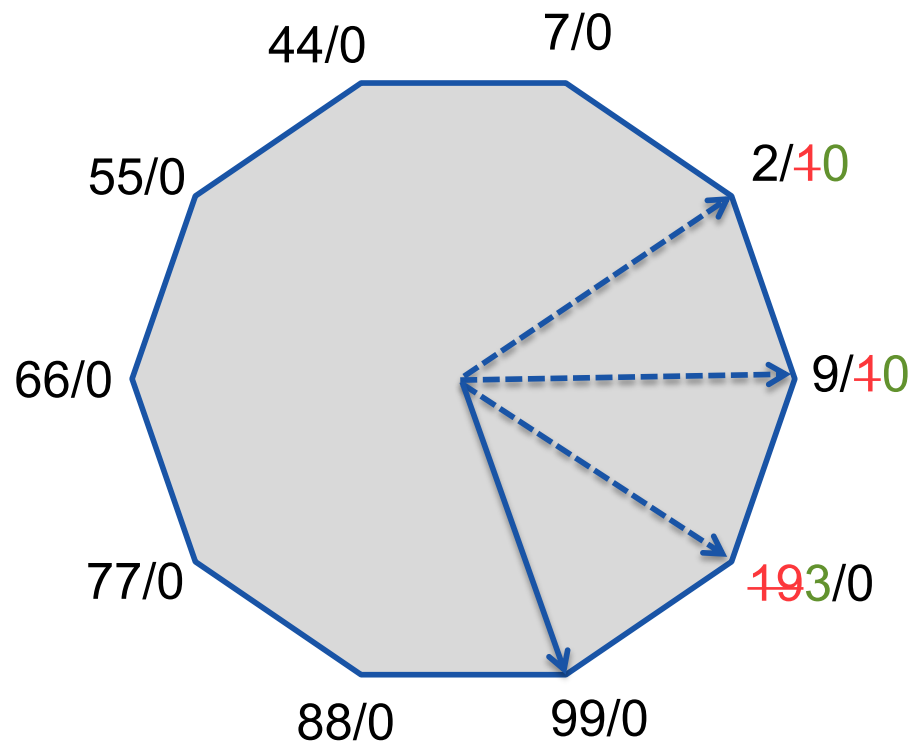
Klockalgoritmen



- Tänk dig sidreferenserna ordnade som en cirkulär kö som talen på en urtavla. Varje sida har en referens-bit som initialt sätt till 0...
- Varje gång en sida refereras sätt referens-biten till 1...
- Read 2, träff, set
- Write 9, träff, set



Klockalgoritmen



- Tänk dig sidreferenserna ordnade som en cirkulär kö som talen på en urtavla. Varje sida har en referens-bit som initialt sätt till 0...
- Varje gång en sida refereras sätt referens-biten till 1...
- När en sida ska slängas ut förs en "visaren" framåt på urtavlan tills den hittar en sida där referensbiten inte är satt, samtidigt som den nollställer referensbiten på de referenser som den passerar!
- Read 2, träff, set
- Write 9, träff, set
- Read 3, miss...
 - 2, RU, nollställ
 - 9, RU, nollställ
 - 19, out, 3 in, ptr 99!

Sidan som "visaren" stannar på slängs ut... ...referenserna "bakom" visaren måste refereras igen innan visaren hinner dit på nytt, annars...



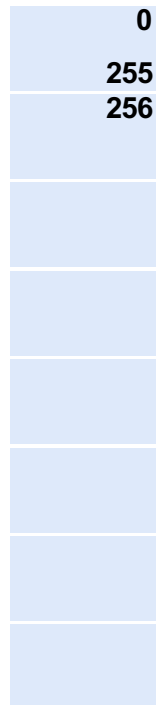
Dirty & Other VM Policies

- Avoid dirty=wr pages if possible
- Page selection
 - Demand paging
 - Prefetching
- Thrashing
 - Admission control
 - Brute force



Målsättning: Testa HIT rate...

VM



Varje process har tillgång till 2kByte adressrymd!

Adressrymden är uppdelade 8 block om 256Byte...

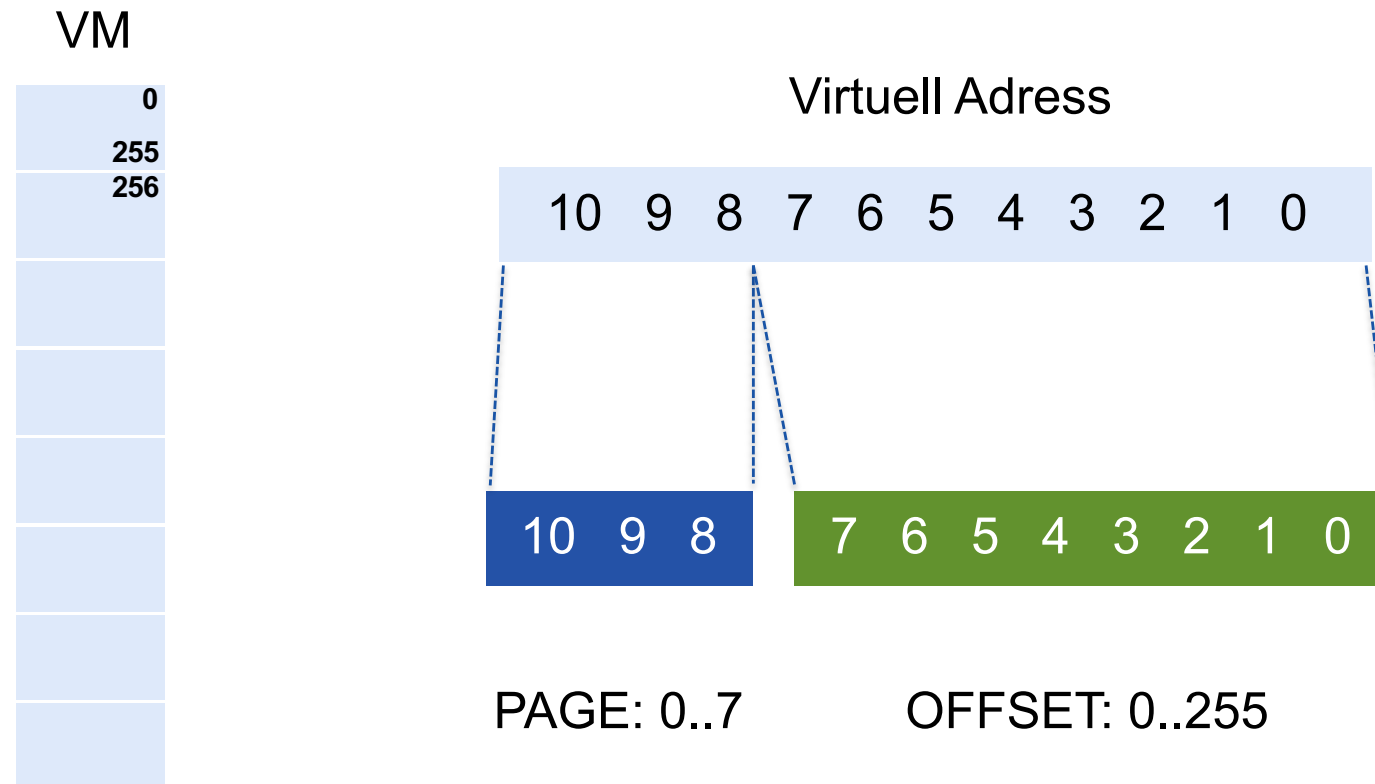
Hur många bitar är en virtuell adress?

Hur många avgör vilken sida det är?

Hur många bitar är offseten?



Målsättning: Testa HIT rate...





Målsättning: Testa HIT rate...

VM

0
255
256

8*256B

Det fysiska minnet har plats
för 16 sidor...

...hur många rader måste
sidtabellen innehålla?

...vilken information finns
på en rad i sidtabellen?

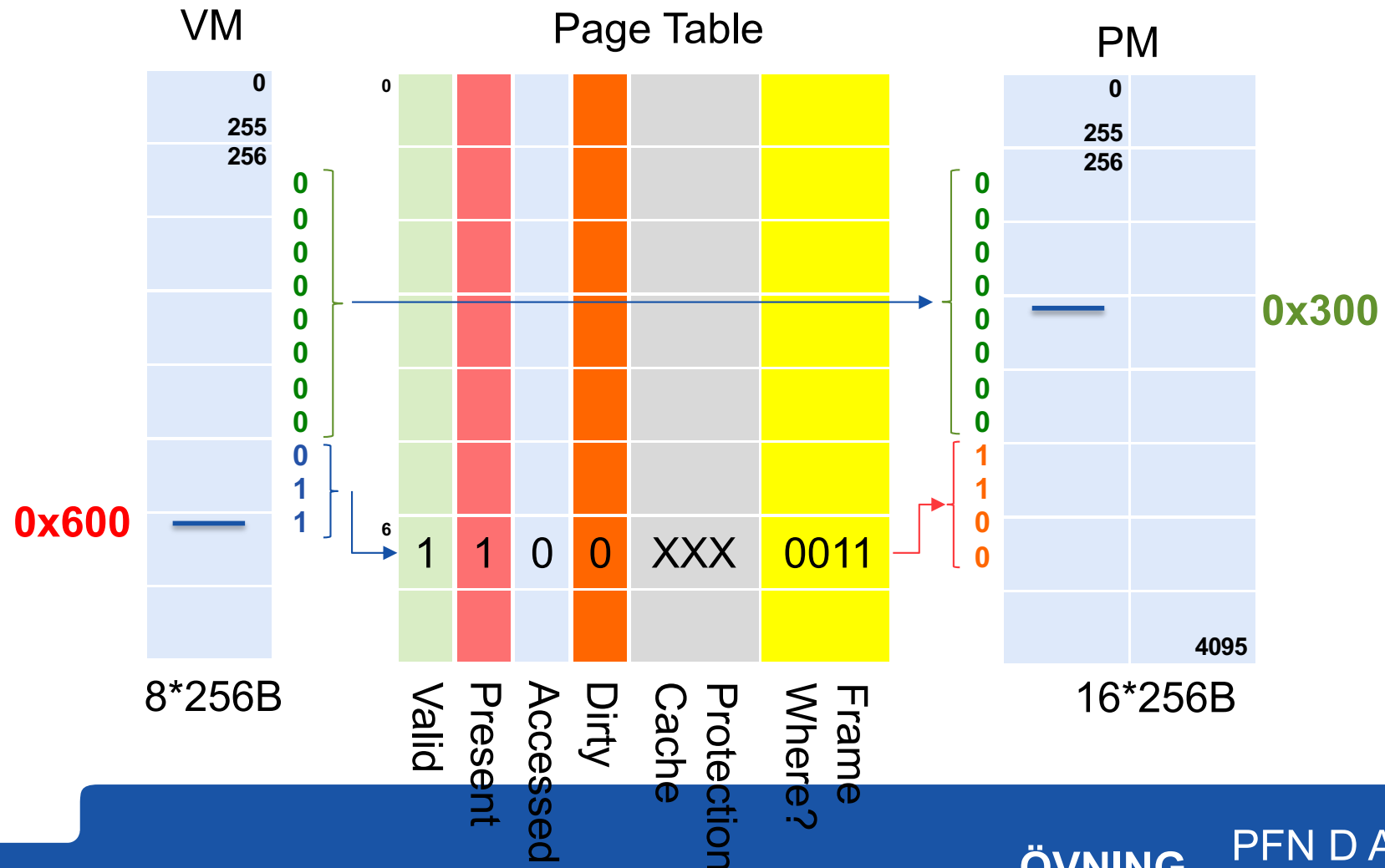
(Tänk STORT!)

PM

0	
255	
256	
	4095

16*256B

Målsättning: Testa HIT rate...



```

#include <stdio.h>

#define PAGESIZE 256
#define VMEMSIZE 8*PAGESIZE
#define PHYSSIZE 2*VMEMSIZE

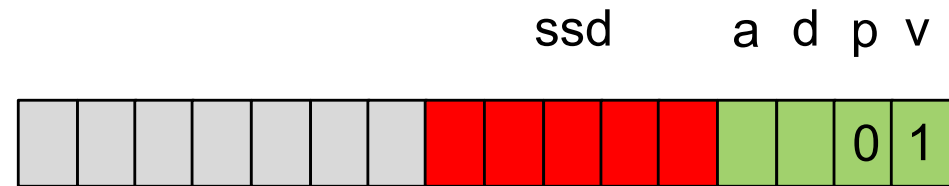
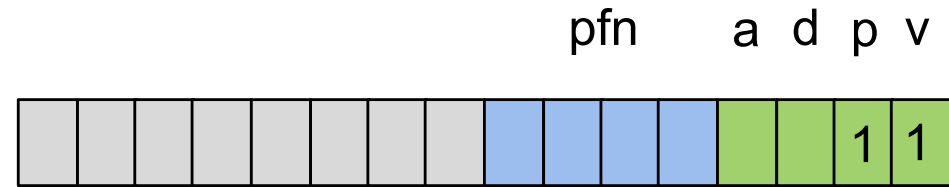
#define PTS 8
typedef struct {
    unsigned short v : 1;
    unsigned short p : 1;
    unsigned short d : 1;
    unsigned short a : 1;
    union {
        unsigned short pfn : 4;
        unsigned short ssd : 5;
    };
} PTEt;

int main(int argc, char **argv){
    PTEt thePT[VMEMSIZE/PAGESIZE];

    printf("MMU 0.1\n");

    printf("MMU Done!\n");
    return 0;
}

```



```
void initValidPTE(PTEt *pPTE, unsigned short pfn){
```

0	1	1	
X	1	0	(Valid men ej present)
X	0	X	
X	0	X	
X	#		
X		#	

```
void initInvalidPTE(PTEt *pPTE){
```

```

#include <stdio.h>

#define PAGESIZE 256
#define VMEMSIZE 8*PAGESIZE
#define PHYSSIZE 2*VMEMSIZE

#define PTS 8
typedef struct {
    unsigned short v : 1;
    unsigned short p : 1;
    unsigned short d : 1;
    unsigned short a : 1;
    union {
        unsigned short pfn : 4;
        unsigned short ssd : 5;
    };
} PTEt;

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0; ←
    printf("MMU 0.1\n");

    printf("MMU Done!\n");
    return 0;
}

```

```

void initValidPTE(PTEt *pPTE, unsigned short pfn){
    pPTE->v = 1;
    pPTE->p = 1;
    pPTE->d = 0;
    pPTE->a = 0;
    pPTE->pfn = pfn;
}

void initInvalidPTE(PTEt *pPTE){
    pPTE->v = 0;
}

int nextFreeFrame(unsigned int * pFT){

```

Sök efter en 0:a upp till 16 ggr...
 ...träff -> sätt biten till 1, returnera positionen!
 ...fullt -> returnera -1!



& = Bitwise AND, << shift left

```

#include <stdio.h>

#define PAGESIZE 256
#define VMEMSIZE 8*PAGESIZE
#define PHYSSIZE 2*VMEMSIZE

#define PTS 8
typedef struct {
    unsigned short v : 1;
    unsigned short p : 1;
    unsigned short d : 1;
    unsigned short a : 1;
    union {
        unsigned short pfn : 4;
        unsigned short ssd : 5;
    };
} PTEt;

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    printf("MMU 0.1\n");
    printf("MMU Done!\n");
    return 0;
}

```

```

void initValidPTE(PTEt *pPTE, unsigned short pfn){
    pPTE->v = 1;
    pPTE->p = 1;
    pPTE->d = 0;
    pPTE->a = 0;
    pPTE->pfn = pfn;
}

void initInvalidPTE(PTEt *pPTE){
    pPTE->v = 0;
}

int nextFreeFrame(unsigned int * pFT){
    for (int i=0; i<PHYSSIZE/PAGESIZE; i++){
        if (!((*pFT)&1<<i)) {
            (*pFT)+=(1<<i);
            return i;
        }
    }
    return -1;
}

```

```

initInvalidPTE(&thePT[0]); //UT: PT/FT
for (int i=1; i<VMEMSIZE/PAGESIZE; i++){
    initValidPTE(&thePT[i], nextFreeFrame(&theFT));
}

```

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;

    printf("MMU 0.2\n");

    initInvalidPTE(&thePT[0]); //UT: PT/FT
    for (int i=1; i<VMEMSIZE/PAGESIZE; i++)
        initValidPTE(&thePT[i], nextFreeFrame(&theFT));
    viewMMU(thePT, theFT);

    printf("MMU Done!\n");
    return 0;
}

```

MMU 0.2							
PN	:	V	P	D	A	F/W	PYS
00	:	I	N	V	A	L I D!	X 0
01	:	1	1	0	0	00	X 0
02	:	1	1	0	0	01	X 0
03	:	1	1	0	0	02	X 0
04	:	1	1	0	0	03	X 0
05	:	1	1	0	0	04	X 0
06	:	1	1	0	0	05	X 0
07	:	1	1	0	0	06	0 0
MMU Done!							

```

void viewMMU(PTEt aPT[], unsigned int aFT){

```

?:	Conditional Expression.	If Condition is true ? then value X : otherwise value Y
----	-------------------------	--

(test) ? 'X' : 'O'

```

}

```



```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;

    printf("MMU 0.2\n");

    initInvalidPTE(&thePT[0]); //UT: PT/FT
    for (int i=1; i<VMEMSIZE/PAGESIZE; i++)
        initValidPTE(&thePT[i], nextFreeFrame(&theFT));
    viewMMU(thePT, theFT);

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.2
PN : V P D A      F/W      PYS
00 : I N V A L I D!      X 0
01 : 1 1 0 0      00      X 0
02 : 1 1 0 0      01      X 0
03 : 1 1 0 0      02      X 0
04 : 1 1 0 0      03      X 0
05 : 1 1 0 0      04      X 0
06 : 1 1 0 0      05      X 0
07 : 1 1 0 0      06      0 0
MMU Done!

```

```

void viewMMU(PTEt aPT[], unsigned int aFT){
    printf("PN : V P D A      F/W\t PYS\n");
    for (int row=0; row<VMEMSIZE/PAGESIZE; row++) {
        if (aPT[row].v) {
            printf("%02d : 1 1 %1d %1d      %02d\t %c %c\n",
                row, aPT[row].d, aPT[row].a, aPT[row].pfn,
                (aFT&1<<row)?'X':'0', (aFT&1<<(row+8))?'X':'0');
        } else {
            printf("%02d : I N V A L I D!\t %c %c\n",
                row,
                (aFT&1<<row)?'X':'0', (aFT&1<<(row+8))?'X':'0');
        }
    }
}

```

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;

    printf("MMU 0.3\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!    X 0
05 : I N V A L I D!    X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

CODE
 DATA
 HEAP
 HEAP
 HEAP
 -
 -
 STACK
 STACK

```

void initMMU(PTEt aPT[], unsigned int *pFT){

```

```

}

```

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;

    printf("MMU 0.3\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!    X 0
05 : I N V A L I D!    X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

CODE
 DATA
 HEAP
 HEAP
 HEAP
 -
 -
 STACK
 STACK

```

void initMMU(PTEt aPT[], unsigned int *pFT){
    for (int row=0; row<VMEMSIZE/PAGESIZE; row++) {
        switch (row) {
            case 0:
                initValidPTE(&aPT[row],nextFreeFrame(pFT)); break;
            case 1: case 2: case 3:
                initValidPTE(&aPT[row],nextFreeFrame(pFT)); break;
            case 4: case 5:
                initInvalidPTE(&aPT[row]); break;
            case 6: case 7:
                initValidPTE(&aPT[row],nextFreeFrame(pFT)); break;
        }
    }
}

```



```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

MMU 0.3

PN	V	P	D	A	F/W	PYS
00	1	1	0	0	00	X 0
01	1	1	0	0	01	X 0
02	1	1	0	0	02	X 0
03	1	1	0	0	03	X 0
04	I	N	V	A	L	I D!
05	I	N	V	A	L	I D!
06	1	1	0	0	04	0 0
07	1	1	0	0	05	0 0

MMU Done!

MISS!

0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
V	PFN				VPN		

0 0 0

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT,
    lupMMU(theTLB, thePT,
    lupMMU(theTLB, thePT,
    lupMMU(theTLB, thePT,
    lupMMU(theTLB, thePT,
    lupMMU(theTLB, thePT,
    lupMMU(theTLB, thePT,

    printf("MMU Done!\n");
    return 0;
}

```

MMU 0.3

PN	:	V	P	D	A	F/W	PYS
00	:	1	1	0	0	00	X 0
01	:	1	1	0	0	01	X 0
02	:	1	1	0	0	02	X 0
03	:	1	1	0	0	03	X 0
04	:	I	N	V	A	L I D!	X 0
05	:	I	N	V	A	L I D!	X 0
06	:	1	1	0	0	04	0 0
07	:	1	1	0	0	05	0 0

MMU Done!

PAGE0); //UT: Valid!! --- TLB MISS
 PAGE4); //UT: Invalid --- TLB MISS
 PAGE6); //UT: Valid!! --- TLB MISS
 PAGE6); //UT: Valid!! *** TLB HIT!
 PAGE7); //UT: Valid!! --- TLB MISS
 PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
 PAGE3); //UT: Valid!! *** TLB HIT!

MISS!

1	0	0	0	0	0	0	0
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
V	PFN				VPN		

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

MISS!

1	0	0	0	0	0	0	0
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
V	PFN				VPN		

1 0 0


```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

MMU 0.3

PN	V	P	D	A	F/W	PYS
00	1	1	0	0	00	X 0
01	1	1	0	0	01	X 0
02	1	1	0	0	02	X 0
03	1	1	0	0	03	X 0
04	INVALID!					X 0
05	INVALID!					X 0
06	1	1	0	0	04	0 0
07	1	1	0	0	05	0 0

MMU Done!

MISS!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
V	PFN				VPN		

Segmentation fault
Process killed

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

MISS!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
0	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-
V	PFN				VPN		

1 1 0

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```
MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!
```

MISS!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	1	0	0	1	1	0
0	-	-	-	-	-	-	-
V	PFN				VPN		


```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

HIT!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	1	0	0	1	1	0
0	-	-	-	-	-	-	-
V	PFN				VPN		

1 1 0


```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

MISS!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	1	0	0	1	1	0
0	-	-	-	-	-	-	-
V	PFN				VPN		

1 1 1

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

MMU 0.3

PN	V	P	D	A	F/W	PYS
00	1	1	0	0	00	X 0
01	1	1	0	0	01	X 0
02	1	1	0	0	02	X 0
03	1	1	0	0	03	X 0
04	I	N	V	A	L I D!	X 0
05	I	N	V	A	L I D!	X 0
06	1	1	0	0	04	0 0
07	1	1	0	0	05	0 0

MMU Done!

MISS!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	1	0	0	1	1	0
1	0	1	0	1	1	1	1
V	PFN				VPN		

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

MISS!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	1	0	0	1	1	0
1	0	1	0	1	1	1	1
V	PFN				VPN		

0 1 1


```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0},

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```
MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!
```

MISS!

Random!
Mapping
6>4 replaced

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	0	1	1	0	1	1
1	0	1	0	1	1	1	1
V	PFN				VPN		

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,

    printf("MMU 0.5\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    lupMMU(theTLB, thePT, PAGE0); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE4); //UT: Invalid --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE6); //UT: Valid!! *** TLB HIT!
    lupMMU(theTLB, thePT, PAGE7); //UT: Valid!! --- TLB MISS
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! +++ TLB MISS/REPLACE
    lupMMU(theTLB, thePT, PAGE3); //UT: Valid!! *** TLB HIT!

    printf("MMU Done!\n");
    return 0;
}

```

```

MMU 0.3
PN : V P D A      F/W      PYS
00 : 1 1 0 0      00      X 0
01 : 1 1 0 0      01      X 0
02 : 1 1 0 0      02      X 0
03 : 1 1 0 0      03      X 0
04 : I N V A L I D!      X 0
05 : I N V A L I D!      X 0
06 : 1 1 0 0      04      0 0
07 : 1 1 0 0      05      0 0
MMU Done!

```

HIT!

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	0	1	1	0	1	1
1	0	1	0	1	1	1	1
V	PFN				VPN		

0 1 1


```

void lupMMU(TLBt aTLB[], PTEt aPT[],int virtAddr){
    int f=-1;
    for (int i=0; i<ATLBSIZE; i++)
        if (aTLB[i].v) {
            if (aTLB[i].vpn==virtAddr/PAGESIZE) {
                printf("VA:%03X PAGE:%01X OFFSET:%02X FRAME:%02X *** Ok\n",
                    virtAddr, virtAddr/PAGESIZE, virtAddr&255, aTLB[i].pfn);
                return;
            }
        } else {
            f=i;
        }

    printf("VA:%03X PAGE:%01X OFFSET:%02X          --- TLB MISS!\n",
        virtAddr, virtAddr/PAGESIZE, virtAddr&255);

    if (f<0) {
        f=rand()%4; //Rnd strategy if cache full!
    }

    aTLB[f].v = 1;
    aTLB[f].vpn = virtAddr/PAGESIZE;
    aTLB[f].pfn = aPT[virtAddr/PAGESIZE].pfn;

    lupMMU(aTLB, aPT, virtAddr);
}

```

HIT?!

MISS

Retry

1	0	0	0	0	0	0	0
1	X	X	X	X	1	0	0
1	0	0	1	1	0	1	1
1	0	1	0	1	1	1	1
V	PFN				VPN		

```

int main(int argc, char **argv){

    PTEt thePT[VMEMSIZE/PAGESIZE];
    unsigned int theFT=0;
    TLBt theTLB[ATLBSIZE]={ {0,0,0}, {0,0,0}, {0,0,0}, {0,0,0} };

    printf("MMU 0.6\n");

    initMMU(thePT, &theFT);
    viewMMU(thePT, theFT);

    int pc = 0;
    int hp = 0;
    int sp = 0;
    int hit= 0;
    for (int i=0; i<TIMES; i++){
        switch (rand()%8) {
            case 0: // 1/8 Chance code access
                hit+=lupMMU(theTLB, thePT, (0+((++pc+gauss(PAGESIZE))%DATA))); break;
            case 1: case 2: case 3: // 3/8 Chance heap access
                hit+=lupMMU(theTLB, thePT, (HEAP+((++hp+gauss(PAGESIZE))%(3*DATA)))); break;
            case 4: case 5: case 6: case 7: // 4/8 Chance stack access
                hit+=lupMMU(theTLB, thePT, (STACK+((++sp+gauss(PAGESIZE))%(2*DATA)))); break;
        }
    }
    printf("Hit rate %3.2f\n", ((float)hit/TIMES)*100.0f);

    printf("MMU Done!\n");
    return 0;
}

```

```

VA:052 PAGE:0 OFFSET:52 FRAME:00 *** 0k
VA:2F5 PAGE:2 OFFSET:F5 FRAME:02 *** 0k
VA:6AC PAGE:6 OFFSET:AC FRAME:04 *** 0k
VA:6B1 PAGE:6 OFFSET:B1 FRAME:04 *** 0k
VA:6AC PAGE:6 OFFSET:AC FRAME:04 *** 0k
VA:2F1 PAGE:2 OFFSET:F1 FRAME:02 *** 0k
VA:6AF PAGE:6 OFFSET:AF FRAME:04 *** 0k
VA:6B3 PAGE:6 OFFSET:B3 FRAME:04 *** 0k
VA:059 PAGE:0 OFFSET:59 FRAME:00 *** 0k
VA:2F5 PAGE:2 OFFSET:F5 FRAME:02 *** 0k
VA:6AE PAGE:6 OFFSET:AE FRAME:04 *** 0k
VA:2F6 PAGE:2 OFFSET:F6 FRAME:02 *** 0k
VA:6B1 PAGE:6 OFFSET:B1 FRAME:04 *** 0k
VA:6B5 PAGE:6 OFFSET:B5 FRAME:04 *** 0k
VA:2F9 PAGE:2 OFFSET:F9 FRAME:02 *** 0k
VA:2F5 PAGE:2 OFFSET:F5 FRAME:02 *** 0k
VA:2FE PAGE:2 OFFSET:FE FRAME:02 *** 0k
VA:05C PAGE:0 OFFSET:5C FRAME:00 *** 0k
VA:6B3 PAGE:6 OFFSET:B3 FRAME:04 *** 0k
VA:2FB PAGE:2 OFFSET:FB FRAME:02 *** 0k
VA:6B5 PAGE:6 OFFSET:B5 FRAME:04 *** 0k
VA:6B6 PAGE:6 OFFSET:B6 FRAME:04 *** 0k
VA:302 PAGE:3 OFFSET:02 --- TLB MISS!
VA:302 PAGE:3 OFFSET:02 FRAME:03 *** 0k
VA:05D PAGE:0 OFFSET:5D --- TLB MISS!
VA:05D PAGE:0 OFFSET:5D FRAME:00 *** 0k
VA:6B5 PAGE:6 OFFSET:B5 FRAME:04 *** 0k
VA:2FD PAGE:2 OFFSET:FD FRAME:02 *** 0k
VA:6B8 PAGE:6 OFFSET:B8 FRAME:04 *** 0k
VA:2FE PAGE:2 OFFSET:FE FRAME:02 *** 0k
VA:058 PAGE:0 OFFSET:58 FRAME:00 *** 0k
VA:6B8 PAGE:6 OFFSET:BB FRAME:04 *** 0k
VA:301 PAGE:3 OFFSET:01 --- TLB MISS!
VA:301 PAGE:3 OFFSET:01 FRAME:03 *** 0k
VA:6B8 PAGE:6 OFFSET:BB FRAME:04 *** 0k
VA:2FE PAGE:2 OFFSET:FE --- TLB MISS!
VA:2FE PAGE:2 OFFSET:FE FRAME:02 *** 0k

```

***10 000**

```

VA:09B PAGE:0 OFFSET:9B FRAME:00 *** 0k
VA:7CF PAGE:7 OFFSET:CF FRAME:05 *** 0k
VA:7D1 PAGE:7 OFFSET:D1 FRAME:05 *** 0k
VA:7D7 PAGE:7 OFFSET:D7 FRAME:05 *** 0k
VA:7D9 PAGE:7 OFFSET:D9 FRAME:05 *** 0k
VA:7D3 PAGE:7 OFFSET:D3 FRAME:05 *** 0k
VA:3B4 PAGE:3 OFFSET:B4 FRAME:03 *** 0k
VA:7D3 PAGE:7 OFFSET:D3 FRAME:05 *** 0k
VA:7D3 PAGE:7 OFFSET:D3 FRAME:05 *** 0k
VA:7D8 PAGE:7 OFFSET:D8 FRAME:05 *** 0k
Hit rate 99.20
MMU Done!

```

“Rimligt!” →



Sammantaget...

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset    = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```




Begrepp

Paging model

Linear Page Table, Address translation

Virtual Page vs. Page Frame

Page Table Base Register

Page, Segmentation, Protection fault

Translation Look-aside buffer (TLB)

Spatial & temporal locality

Least recently used vs. Random

Multi-level Page Table

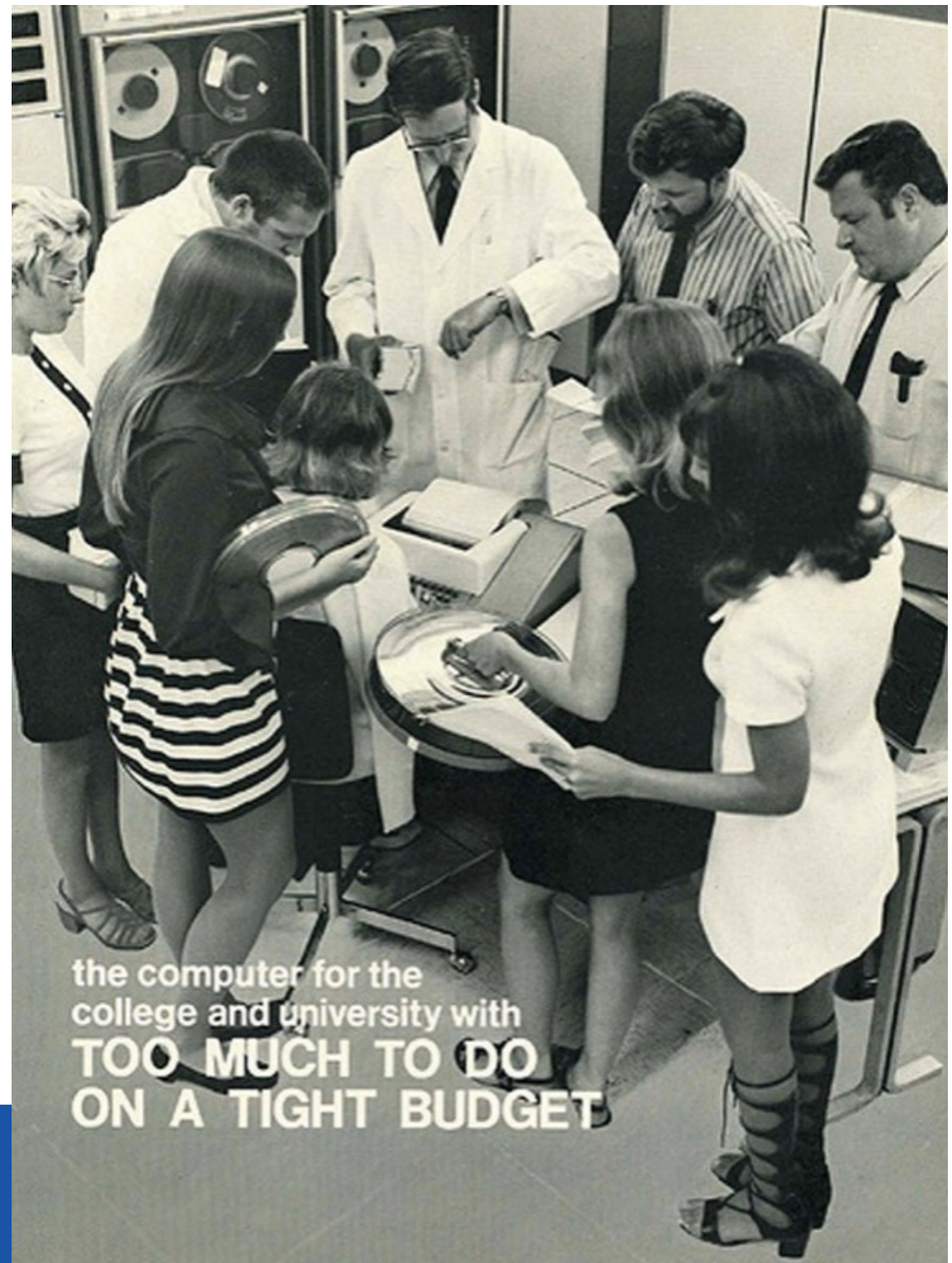
Storage Hierarchies

Swap space

Page replacement policies

FIFO vs. Random vs. LRU/LFU

The Clock Algorithm





Historik

1810xx AC 0.8 Skapad, huvuddrag.

190110 AC 1.0 Version 1.0.

2104xx AC 1.1 Mindre uppdateringar och förbättringar.

220412 AC 1.2 Mindre uppdateringar.